VERILOG HDL IMPLEMENTATION OF A 5-STAGE PIPELINED RISC PROCESSOR

B. Tech Internship Project Work Report | RTL Design & Verification

Student: Chella Jeevana Jyothi

Branch: B.Tech VLSI Technology (7th Sem)

Email: chellajeevanajyothi@gmail.com

Institution: SRK Institute of Technology, Vijayawada

Affiliation: JNTU Kakinada

GitHub Repo: RTL-Design-with-Verilog-HDL

1. EXECUTIVE SUMMARY & MICROARCHITECTURE

This project presents the complete RTL design, Verilog implementation, and functional verification of a 32-bit 5-stage pipelined RISC processor. The microarchitecture divides instruction processing into five pipeline stages—**Instruction Fetch (IF)**, **Instruction Decode (ID)**, **Execute (EX)**, **Memory Access (MEM)**, and **Write Back (WB)**—increasing throughput and allowing multi-instruction concurrency.

5-Stage Pipeline Architecture Overview

Pipeline Stages Breakdown

Stage	Primary Operations	Key Submodules Involved
1. IF (Fetch)	Fetches instruction from memory based on PC; increments PC by +4.	PC Register, Instruction Memory (ROM)
2. ID (Decode)	Decodes opcode/funct, reads register operands, generates control signals, sign-extends immediate.	Register File, Control Unit, Hazard Unit
3. EX (Execute)	Performs arithmetic/logical ops, computes branch targets, selects forwarded ALU operands.	ALU, ALU Control Unit, Forwarding Unit
4. MEM (Memory)	Reads or writes payload from/to Data Memory for LW/SW instructions. Evaluates branch outcomes.	Data Memory (RAM), Branch Logic
5. WB (Write Back)	Writes computed ALU results or loaded memory data back into target register.	Write-Back Multiplexer

2. HAZARD DETECTION & RESOLUTION STRATEGY

Pipeline Hazards Addressed:

- **Data Hazards (RAW - Read After Write):** Resolved via an execution-stage **Forwarding Unit** which bypasses computation results directly from EX/MEM or MEM/WB pipeline registers to the ALU inputs without stalling.
- **Load-Use Hazards:** Detected by the **Hazard Detection Unit**. When an instruction immediately depends on data fetched by an active `LW` instruction, a 1-cycle stall is inserted, freezing the PC and IF/ID registers and injecting a NOP into ID/EX.

- **Control Hazards (Branches):** Resolved by flushing the `IF/ID` register whenever a branch condition is evaluated as taken in the MEM stage.

3. COMPLETE VERILOG HDL CORE (`PIPELINED_RISC.V`)

Top Module: pipelined_risc.v

```
`timescale 1ns / 1ps

// ============================================================================
// 32-BIT 5-STAGE PIPELINED RISC PROCESSOR CORE
// Author: Chella Jeevana Jyothi
// ============================================================================

module pipelined_risc (
    input  wire        clk,
    input  wire        rst,
    output wire [31:0] wb_result
);

    // ------------------------------------------------------------------------
    // 1. INSTRUCTION FETCH (IF) STAGE
    // ------------------------------------------------------------------------
    reg  [31:0] pc;
    wire [31:0] pc_next, pc_plus_4;
    wire [31:0] if_instr;
    wire        pc_write, if_id_write, if_id_flush;

    assign pc_plus_4 = pc + 32'd4;

    always @(posedge clk or posedge rst) begin
        if (rst)
            pc <= 32'h0000_0000;
        else if (pc_write)
            pc <= pc_next;
    end

    instruction_memory imem (
        .addr(pc),
        .instr(if_instr)
    );

    // IF/ID Pipeline Register
    reg [31:0] if_id_pc_plus_4, if_id_instr;

    always @(posedge clk or posedge rst) begin
        if (rst || if_id_flush) begin
            if_id_pc_plus_4 <= 32'b0;
            if_id_instr     <= 32'b0;
        end else if (if_id_write) begin
            if_id_pc_plus_4 <= pc_plus_4;
            if_id_instr     <= if_instr;
        end
    end

    // ------------------------------------------------------------------------
    // 2. INSTRUCTION DECODE (ID) STAGE
    // ------------------------------------------------------------------------
    wire [5:0]  opcode    = if_id_instr[31:26];
    wire [4:0]  rs        = if_id_instr[25:21];
    wire [4:0]  rt        = if_id_instr[20:16];
    wire [4:0]  rd        = if_id_instr[15:11];
    wire [15:0] imm16     = if_id_instr[15:0];
```

```
    wire [31:0] id_rdata1, id_rdata2, id_imm_ext;
    wire [31:0] wb_wdata;
    wire [4:0]  mem_wb_rd;
    wire        mem_wb_reg_write;

    assign id_imm_ext = {{16{imm16[15]}}, imm16};

    register_file regfile (
        .clk(clk), .rst(rst),
        .raddr1(rs), .rdata1(id_rdata1),
        .raddr2(rt), .rdata2(id_rdata2),
        .waddr(mem_wb_rd), .wdata(wb_wdata), .wen(mem_wb_reg_write)
    );

    wire reg_dst, alu_src, mem_to_reg, reg_write, mem_read, mem_write, branch;
    wire [1:0] alu_op;

    control_unit ctrl (
        .opcode(opcode), .reg_dst(reg_dst), .alu_src(alu_src),
        .mem_to_reg(mem_to_reg), .reg_write(reg_write),
        .mem_read(mem_read), .mem_write(mem_write),
        .branch(branch), .alu_op(alu_op)
    );

    wire id_ex_mem_read;
    wire [4:0] id_ex_rt;
    wire stall;

    hazard_detection_unit hazard_unit (
        .id_ex_mem_read(id_ex_mem_read), .id_ex_rt(id_ex_rt),
        .if_id_rs(rs), .if_id_rt(rt),
        .pc_write(pc_write), .if_id_write(if_id_write), .stall(stall)
    );

    // ID/EX Pipeline Register
    reg [31:0] id_ex_pc_plus_4, id_ex_rdata1, id_ex_rdata2, id_ex_imm_ext;
    reg [4:0]  id_ex_rs, id_ex_rt_reg, id_ex_rd;
    reg        id_ex_reg_dst, id_ex_alu_src, id_ex_mem_to_reg, id_ex_reg_write;
    reg        id_ex_mem_read_reg, id_ex_mem_write, id_ex_branch;
    reg [1:0]  id_ex_alu_op;

    assign id_ex_mem_read = id_ex_mem_read_reg;
    assign id_ex_rt       = id_ex_rt_reg;

    always @(posedge clk or posedge rst) begin
        if (rst || stall) begin
            id_ex_reg_dst      <= 1'b0; id_ex_alu_src      <= 1'b0;
            id_ex_mem_to_reg   <= 1'b0; id_ex_reg_write    <= 1'b0;
            id_ex_mem_read_reg <= 1'b0; id_ex_mem_write    <= 1'b0;
            id_ex_branch       <= 1'b0; id_ex_alu_op       <= 2'b0;
            id_ex_pc_plus_4    <= 32'b0; id_ex_rdata1       <= 32'b0;
            id_ex_rdata2       <= 32'b0; id_ex_imm_ext      <= 32'b0;
            id_ex_rs           <= 5'b0;  id_ex_rt_reg       <= 5'b0;
            id_ex_rd           <= 5'b0;
        end else begin
            id_ex_reg_dst      <= reg_dst;   id_ex_alu_src      <= alu_src;
            id_ex_mem_to_reg   <= mem_to_reg;id_ex_reg_write    <= reg_write;
            id_ex_mem_read_reg <= mem_read;  id_ex_mem_write    <= mem_write;
            id_ex_branch       <= branch;    id_ex_alu_op       <= alu_op;
            id_ex_pc_plus_4    <= if_id_pc_plus_4; id_ex_rdata1 <= id_rdata1;
            id_ex_rdata2       <= id_rdata2; id_ex_imm_ext      <= id_imm_ext;
            id_ex_rs           <= rs;        id_ex_rt_reg       <= rt;
            id_ex_rd           <= rd;
        end
    end

    // ---------------------------------------------------------------------
```

```
    // 3. EXECUTE (EX) STAGE
    // ------------------------------------------------------------------------
    wire [1:0] forward_a, forward_b;
    reg  [31:0] alu_in_a, alu_in_b_raw, alu_in_b;
    wire [3:0]  alu_control;
    wire [31:0] ex_alu_result;
    wire        ex_zero;
    wire [4:0]  ex_write_reg;
    wire [31:0] ex_branch_target;

    always @(*) begin
        case (forward_a)
            2'b00: alu_in_a = id_ex_rdata1;
            2'b10: alu_in_a = ex_mem_alu_result;
            2'b01: alu_in_a = wb_wdata;
            default: alu_in_a = id_ex_rdata1;
        endcase
    end

    always @(*) begin
        case (forward_b)
            2'b00: alu_in_b_raw = id_ex_rdata2;
            2'b10: alu_in_b_raw = ex_mem_alu_result;
            2'b01: alu_in_b_raw = wb_wdata;
            default: alu_in_b_raw = id_ex_rdata2;
        endcase
    end

    always @(*) alu_in_b = id_ex_alu_src ? id_ex_imm_ext : alu_in_b_raw;

    assign ex_write_reg     = id_ex_reg_dst ? id_ex_rd : id_ex_rt_reg;
    assign ex_branch_target = id_ex_pc_plus_4 + (id_ex_imm_ext << 2);

    alu_control_unit alu_ctrl
(.alu_op(id_ex_alu_op), .funct(id_ex_imm_ext[5:0]), .alu_control(alu_control));
    alu alu_inst
(.a(alu_in_a), .b(alu_in_b), .alu_control(alu_control), .result(ex_alu_result), .zero(ex_zero));

    forwarding_unit fwd_unit (
        .id_ex_rs(id_ex_rs), .id_ex_rt(id_ex_rt_reg),
        .ex_mem_rd(ex_mem_rd), .ex_mem_reg_write(ex_mem_reg_write),
        .mem_wb_rd(mem_wb_rd), .mem_wb_reg_write(mem_wb_reg_write),
        .forward_a(forward_a), .forward_b(forward_b)
    );

    // EX/MEM Pipeline Register
    reg [31:0] ex_mem_branch_target, ex_mem_alu_result, ex_mem_write_data;
    reg [4:0]  ex_mem_rd;
    reg        ex_mem_zero, ex_mem_branch, ex_mem_mem_read, ex_mem_mem_write, ex_mem_reg_write,
ex_mem_mem_to_reg;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            ex_mem_branch_target <= 32'b0; ex_mem_alu_result <= 32'b0;
            ex_mem_write_data <= 32'b0;    ex_mem_rd         <= 5'b0;
            ex_mem_zero <= 1'b0;           ex_mem_branch     <= 1'b0;
            ex_mem_mem_read <= 1'b0;       ex_mem_mem_write  <= 1'b0;
            ex_mem_reg_write <= 1'b0;      ex_mem_mem_to_reg <= 1'b0;
        end else begin
            ex_mem_branch_target <= ex_branch_target; ex_mem_alu_result <= ex_alu_result;
            ex_mem_write_data    <= alu_in_b_raw;     ex_mem_rd         <= ex_write_reg;
            ex_mem_zero          <= ex_zero;          ex_mem_branch     <= id_ex_branch;
            ex_mem_mem_read      <= id_ex_mem_read_reg; ex_mem_mem_write <= id_ex_mem_write;
            ex_mem_reg_write     <= id_ex_reg_write;  ex_mem_mem_to_reg <= id_ex_mem_to_reg;
        end
    end

    // ------------------------------------------------------------------------
```

```
    // 4. MEMORY ACCESS (MEM) STAGE
    // ------------------------------------------------------------------------
    wire [31:0] mem_read_data;
    wire        take_branch = ex_mem_branch & ex_mem_zero;

    assign pc_next     = take_branch ? ex_mem_branch_target : pc_plus_4;
    assign if_id_flush = take_branch;

    data_memory dmem
(.clk(clk), .addr(ex_mem_alu_result), .wdata(ex_mem_write_data), .ren(ex_mem_mem_read), .wen(ex_mem_me
m_write), .rdata(mem_read_data));

    // MEM/WB Pipeline Register
    reg [31:0] mem_wb_read_data, mem_wb_alu_result;
    reg [4:0]  mem_wb_rd_reg;
    reg        mem_wb_reg_write_reg, mem_wb_mem_to_reg;

    assign mem_wb_rd        = mem_wb_rd_reg;
    assign mem_wb_reg_write = mem_wb_reg_write_reg;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            mem_wb_read_data    <= 32'b0; mem_wb_alu_result    <= 32'b0;
            mem_wb_rd_reg       <= 5'b0;  mem_wb_reg_write_reg <= 1'b0;
            mem_wb_mem_to_reg   <= 1'b0;
        end else begin
            mem_wb_read_data    <= mem_read_data;     mem_wb_alu_result   <= ex_mem_alu_result;
            mem_wb_rd_reg       <= ex_mem_rd;         mem_wb_reg_write_reg<= ex_mem_reg_write;
            mem_wb_mem_to_reg   <= ex_mem_mem_to_reg;
        end
    end

    // ------------------------------------------------------------------------
    // 5. WRITE BACK (WB) STAGE
    // ------------------------------------------------------------------------
    assign wb_wdata  = mem_wb_mem_to_reg ? mem_wb_read_data : mem_wb_alu_result;
    assign wb_result = wb_wdata;

endmodule
```

4. SUPPORT SUBMODULES & HAZARD UNITS

Submodules: Hazard Detection & Forwarding Units

```
module hazard_detection_unit (
    input  wire       id_ex_mem_read,
    input  wire [4:0] id_ex_rt,
    input  wire [4:0] if_id_rs,
    input  wire [4:0] if_id_rt,
    output reg        pc_write,
    output reg        if_id_write,
    output reg        stall
);
    always @(*) begin
        if (id_ex_mem_read && ((id_ex_rt == if_id_rs) || (id_ex_rt == if_id_rt))) begin
            pc_write    = 1'b0; // Lock PC
            if_id_write = 1'b0; // Lock IF/ID
            stall       = 1'b1; // Inject Bubble
        end else begin
            pc_write    = 1'b1;
            if_id_write = 1'b1;
            stall       = 1'b0;
        end
    end
```

```
endmodule

module forwarding_unit (
    input  wire [4:0] id_ex_rs, id_ex_rt,
    input  wire [4:0] ex_mem_rd, mem_wb_rd,
    input  wire       ex_mem_reg_write, mem_wb_reg_write,
    output reg  [1:0] forward_a, forward_b
);
    always @(*) begin
        if (ex_mem_reg_write && (ex_mem_rd != 0) && (ex_mem_rd == id_ex_rs))
            forward_a = 2'b10;
        else if (mem_wb_reg_write && (mem_wb_rd != 0) && (mem_wb_rd == id_ex_rs))
            forward_a = 2'b01;
        else
            forward_a = 2'b00;

        if (ex_mem_reg_write && (ex_mem_rd != 0) && (ex_mem_rd == id_ex_rt))
            forward_b = 2 me 2'b10;
        else if (mem_wb_reg_write && (mem_wb_rd != 0) && (mem_wb_rd == id_ex_rt))
            forward_b = 2'b01;
        else
            forward_b = 2'b00;
    end
endmodule

module register_file (
    input  wire        clk, rst,
    input  wire [4:0]  raddr1, raddr2, waddr,
    output wire [31:0] rdata1, rdata2,
    input  wire [31:0] wdata,
    input  wire        wen
);
    reg [31:0] registers [0:31];
    integer i;
    assign rdata1 = (raddr1 == 0) ? 32'b0 : registers[raddr1];
    assign rdata2 = (raddr2 == 0) ? 32'b0 : registers[raddr2];

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < 32; i = i + 1) registers[i] <= 32'b0;
        end else if (wen && (waddr != 0)) begin
            registers[waddr] <= wdata;
        end
    end
endmodule

module alu (
    input  wire [31:0] a, b,
    input  wire [3:0]  alu_control,
    output reg  [31:0] result,
    output wire        zero
);
    always @(*) begin
        case (alu_control)
            4'b0000: result = a & b;
            4'b0001: result = a | b;
            4'b0010: result = a + b;
            4'b0110: result = a - b;
            4'b0111: result = (a < b) ? 1 : 0;
            4'b1100: result = ~(a | b);
            default: result = 32'b0;
        endcase
    end
    assign zero = (result == 32'b0);
endmodule
```

5. VERIFICATION SETUP & TESTBENCH

The core is verified using Icarus Verilog and GTKWave. A stimulus testbench populates instruction memory with machine code instructions designed to explicitly trigger data hazards and branch condition evaluations.

Testbench Module: tb_pipelined_risc.v

```
`timescale 1ns / 1ps

module tb_pipelined_risc;
    reg clk;
    reg rst;
    wire [31:0] wb_result;

    pipelined_risc uut (.clk(clk), .rst(rst), .wb_result(wb_result));

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1;
        #20; rst = 0;

        $display("--- Starting Pipelined RISC Simulation ---");
        $monitor("Time=%0t ns | PC=%h | WB Result=%d", $time, uut.pc, wb_result);

        #200;
        if (uut.regfile.registers[3] == 32'd15)
            $display("[PASS] Register R3 = 15 (RAW Hazard Resolved via Forwarding)");
        else
            $display("[FAIL] Register R3 = %d (Expected 15)", uut.regfile.registers[3]);

        $finish;
    end

    initial begin
        $dumpfile("pipeline_wave.vcd");
        $dumpvars(0, tb_pipelined_risc);
    end
endmodule
```

6. SIMULATION RESULTS & CONCLUSION

Functional simulation confirmed that the 5-stage pipelined processor successfully handles sequential instruction execution at 1 instruction per cycle (CPI ≈ 1 under non-hazard conditions). The **Forwarding Unit** eliminated data stalls for back-to-back arithmetic instructions, while the **Hazard Detection Unit** correctly preserved pipeline state during memory read hazards.

Repository Reference: All source files and simulation scripts have been published to GitHub at `https://github.com/chellajeevanajyothi-maker/RTL-Design-with-Verilog-HDL`.